Mini-Projet 1

Compte rendu : infrastructure réseau et serveur

Système de surveillance de la salle 215

Liam Brassat (L.B.)
Tâche 3 : infrastructure réseau et serveur
Octobre 2026

Sommaire

1. Introduction..... 3
   1.1 Ma mission 3
   1.2 Identifiants masqués 3
   1.3 Résultat obtenu 3
2. Matériel et logiciels.....3
3. Configuration réseau.....4
   3.1 Plan d'adressage 4
   3.2 Ports utilisés sur le serveur 4
   3.3 Pourquoi l'adresse a changé 4
4. Mise en place du serveur.....4
   4.1 Préparation de la carte SD 4
   4.2 Connexion à distance 5
   4.3 Installation des paquets 5
   4.4 Adresse IP fixe 5
   4.5 Base de données MariaDB 5
   4.6 Organisation des fichiers 5
   4.7 API Flask 6
   4.8 Démarrage automatique avec systemd 6
   4.9 phpMyAdmin 6
   4.10 Stockage des photos, galerie et tableau de bord 6
5. Base de données.....7
6. API.....7
   6.1 Format attendu 8
   6.2 Codes de réponse 8
7. Raccordement du capteur et de la caméra.....8
   7.1 Capteur PIR (ESP32) 8
   7.2 Caméra 8
8. Tests et résultats..... 8
9. Problèmes rencontrés et solutions..... 9
10. Accès, sécurité et sauvegarde.....10
   10.1 Comptes 10
   10.2 Sécurité 10
   10.3 Sauvegarde 10
11. Choix techniques.....10
12. Bilan..... 11
   12.1 Ce qui fonctionne 11
   12.2 Améliorations possibles 11
   12.3 Ce que j'ai appris 11

1. Introduction

Le Mini-Projet 1 consiste à surveiller la salle 215 avec un capteur de présence et une caméra. J'étais chargé de la tâche 3 : mettre en place le serveur qui reçoit, stocke et affiche toutes les données du système.

1.1 Ma mission

- Installer et configurer un serveur Linux sur un Raspberry Pi 3.
- Lui donner une adresse IP fixe sur le réseau Wi-Fi de la salle.
- Créer une base de données pour stocker les détections et les photos.
- Développer une API web qui reçoit les données du capteur et de la caméra.
- Rendre les données lisibles : phpMyAdmin, tableau de bord et galerie de photos.

1.2 Identifiants masqués

Pour des raisons de sécurité, aucun identifiant réel n'apparaît dans ce document. Ils sont remplacés par des repères entre chevrons :

Repère	Signification
`<UTILISATEUR_PI>`	Compte Linux du Raspberry Pi
`<UTILISATEUR_BDD>`	Compte MariaDB utilisé par l'API
`<MOT_DE_PASSE_BDD>`	Mot de passe de ce compte
`<ADMIN_BDD>`	Compte administrateur MariaDB pour phpMyAdmin
`<MOT_DE_PASSE_ADMIN>`	Mot de passe de ce compte

Les valeurs réelles sont stockées uniquement sur le serveur, dans le fichier `.env` (droits 600), qui n'est jamais publié sur GitHub.

1.3 Résultat obtenu

Le serveur fonctionne : une détection du capteur ou une photo de la caméra est enregistrée dans la base et apparaît sur le tableau de bord en moins de 5 secondes. L'API démarre toute seule avec le Raspberry Pi.

2. Matériel et logiciels

Le serveur est un Raspberry Pi 3, administré à distance depuis un PC Windows.

Élément	Version ou modèle	Rôle
Raspberry Pi 3	0,9 Go de RAM, carte microSD	Serveur central
Raspberry Pi OS	32 bits, Python 3.13	Système d'exploitation
MariaDB	Paquet `mariadb-server`	Base de données
Flask	2.3.2, dans un `venv`	API web en Python
mysql-connector-python	8.0.33	Connexion de Flask à la base
python-dotenv	1.0.0	Lecture du fichier `.env`
Apache 2 + PHP	`apache2`, `php`	Serveur web pour phpMyAdmin
phpMyAdmin	Paquet `phpmyadmin`	Interface web de la base
systemd	Service `mp1-api`	Démarrage automatique de l'API
NetworkManager	Commande `nmcli`	Adresse IP fixe

Élément	Version ou modèle	Rôle
Raspberry Pi Imager	Sur le PC	Installation de l'OS
MobaXterm	26.5	Terminal SSH et transfert de fichiers

3. Configuration réseau

Le Raspberry Pi est connecté au Wi-Fi `Wifi_Iot215` (réseau 172.16.10.0/24) avec l'adresse fixe 172.16.10.239. Le schéma réseau est fourni dans un fichier séparé.

3.1 Plan d'adressage

Appareil	Adresse IP	Attribution
Passerelle (routeur Wi-Fi)	172.16.10.254	Fixe
Raspberry Pi 3 (mon serveur)	**172.16.10.239 /24**	**Fixe, configurée avec nmcli**
ESP32 + capteur PIR	172.16.10.235	DHCP
PC du groupe vidéo	172.16.10.x	DHCP
Caméra IP (reliée au PC vidéo)	192.168.1.12	Réseau séparé, câble Ethernet

Réglages : SSID `Wifi_Iot215`, masque 255.255.255.0 (/24), passerelle 172.16.10.254, DNS 172.16.10.254 et 8.8.8.8.

3.2 Ports utilisés sur le serveur

Port	Service	Utilisé par
22/tcp	SSH	MobaXterm (administration)
80/tcp	Apache et phpMyAdmin	Navigateur web
5000/tcp	API Flask, tableau de bord, galerie	Capteur, caméra, navigateur
3306/tcp	MariaDB	Uniquement en local sur le Pi

3.3 Pourquoi l'adresse a changé

Le plan de départ prévoyait 172.16.10.100. Le fichier `/etc/dhcpcd.conf` a été modifié, mais il est ignoré par les versions récentes de Raspberry Pi OS, qui utilisent NetworkManager. Le Pi recevait donc une adresse différente à chaque réseau (172.20.10.14 sur le partage de connexion du téléphone, puis 172.16.10.239). J'ai fixé l'adresse 172.16.10.239 avec `nmcli`, car le capteur était déjà programmé avec cette adresse.

4. Mise en place du serveur

4.1 Préparation de la carte SD

Raspberry Pi Imager a installé Raspberry Pi OS 32 bits. Dans les options avancées, j'ai réglé avant le premier démarrage :

- le nom d'utilisateur `<UTILISATEUR_PI>` et son mot de passe ;
- le réseau Wi-Fi et son mot de passe ;
- l'activation du SSH ;

- le pays FR, le fuseau horaire Europe/Paris et le clavier fr.

4.2 Connexion à distance

MobaXterm se connecte au Pi en SSH sur le port 22 (hôte `raspberrypi.local`, puis `172.16.10.239`, utilisateur `<UTILISATEUR_PI>`). Son panneau Sftp sert à copier des fichiers entre le PC et le Pi par glisser-déposer.

4.3 Installation des paquets

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3 python3-pip python3-venv
sudo apt install -y mariadb-server git curl wget nano htop
```

MySQL n'est pas disponible pour Raspberry Pi OS : j'ai installé MariaDB, qui est compatible.

4.4 Adresse IP fixe

Avant de fixer l'adresse, j'ai relevé le nom de la connexion et la passerelle :

```
nmcli con show        # nom de la connexion : Wifi_Iot215
ip route              # default via 172.16.10.254
```

Puis j'ai appliqué la configuration manuelle :

```
sudo nmcli con mod "Wifi_Iot215" ipv4.method manual \
  ipv4.addresses 172.16.10.239/24 \
  ipv4.gateway 172.16.10.254 \
  ipv4.dns "172.16.10.254 8.8.8.8"
sudo nmcli con up "Wifi_Iot215"
```

4.5 Base de données MariaDB

L'administrateur `root` de MariaDB ne se connecte qu'avec `sudo`. J'ai créé la base et un utilisateur dédié à l'application :

```
sudo mysql -u root
CREATE DATABASE mp1_surveillance;
CREATE USER '<UTILISATEUR_BDD>'@'localhost' IDENTIFIED BY '<MOT_DE_PASSE_BDD>';
GRANT ALL PRIVILEGES ON mp1_surveillance.* TO '<UTILISATEUR_BDD>'@'localhost';
FLUSH PRIVILEGES;
```

Les tables sont décrites dans le fichier `database/schema.sql`, importé avec :

```
mysql -u <UTILISATEUR_BDD> -p mp1_surveillance < ~/mp1-infrastructure/database/schema.sql
```

Le script `database/init-db.sh` refait toutes ces opérations en une commande.

4.6 Organisation des fichiers

```
~/mp1-infrastructure/
├── server/
│   ├── api/
│   │   ├── app.py            API Flask
│   │   ├── images.py         réception et galerie des photos
│   │   ├── dashboard.py      tableau de bord
│   │   ├── requirements.txt  dépendances Python
│   │   ├── .env              réglages de connexion (hors GitHub)
│   │   └── venv/             environnement Python
│   └── uploads/images/       photos reçues
├── database/
│   ├── schema.sql
│   └── init-db.sh
├── docs/
└── tests/
```

4.7 API Flask

L'API est écrite en Python avec Flask. Ses dépendances sont installées dans un environnement virtuel pour ne pas toucher au Python du système :

```
cd ~/mp1-infrastructure/server/api
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Les réglages de connexion sont lus dans le fichier `.env`, pour ne pas écrire le mot de passe dans le code :

```
DB_HOST=localhost
DB_USER=<UTILISATEUR_BDD>
DB_PASSWORD=<MOT_DE_PASSE_BDD>
DB_NAME=mp1_surveillance
API_PORT=5000
API_HOST=0.0.0.0
```

4.8 Démarrage automatique avec systemd

Lancée à la main, l'API s'arrêtait dès la fermeture du terminal. J'ai créé le service `/etc/systemd/system/mp1-api.service` :

```
[Unit]
Description=MP1 API Flask
After=network-online.target mariadb.service
Wants=network-online.target

[Service]
User=<UTILISATEUR_PI>
WorkingDirectory=/home/<UTILISATEUR_PI>/mp1-infrastructure/server/api
ExecStart=/home/<UTILISATEUR_PI>/mp1-infrastructure/server/api/venv/bin/python app.py
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Puis je l'ai activé :

```
sudo systemctl daemon-reload
sudo systemctl enable --now mp1-api
sudo systemctl status mp1-api      # doit afficher active (running)
```

L'API démarre maintenant avec le Pi et redémarre seule en cas d'arrêt. Après une modification du code : `sudo systemctl restart mp1-api`.

4.9 phpMyAdmin

Le professeur demandait une base gérée avec XAMPP. XAMPP n'existe pas pour le processeur ARM du Raspberry Pi : j'ai donc installé séparément les mêmes logiciels (Apache, PHP, phpMyAdmin).

```
sudo apt install -y apache2 php libapache2-mod-php php-mysql php-mbstring phpmyadmin
```

Pendant l'installation, j'ai coché `apache2` comme serveur web et accepté la configuration avec dbconfig-common. Comme `root` ne peut pas se connecter à phpMyAdmin, j'ai créé un compte administrateur :

```
sudo mysql -u root -e "CREATE USER '<ADMIN_BDD>'@'localhost' IDENTIFIED BY
'<MOT_DE_PASSE_ADMIN>';
GRANT ALL PRIVILEGES ON *.* TO '<ADMIN_BDD>'@'localhost' WITH GRANT OPTION;
FLUSH PRIVILEGES;"
```

phpMyAdmin est accessible sur `http://172.16.10.239/phpmyadmin`.

4.10 Stockage des photos, galerie et tableau de bord

J'ai ajouté deux modules à l'API :

- `images.py` reçoit les photos de la caméra, les range dans `server/uploads/images/` avec un nom unique (date, heure et caméra) et les enregistre dans la base. Dans un navigateur, `/api/images` affiche une galerie : un clic agrandit la photo avec sa date et son heure.
- `dashboard.py` affiche sur `http://172.16.10.239:5000/` l'état de la salle (vert si libre, rouge si occupée), le nombre de détections du jour, un graphique des mouvements heure par heure et les dernières photos. La page se met à jour toutes les 5 secondes.

5. Base de données

La base `mp1_surveillance` contient 6 tables. Chaque enregistrement est horodaté automatiquement par la colonne `timestamp`.

Table	Contenu	Rôle
`pir_events`	`sensor_id`, `timestamp`, `duration`, `status` (active ou inactive)	Détections du capteur PIR
`video_events`	`camera_id`, `event_type`, `timestamp`, `image_path`, `description`	Événements de la caméra
`images`	`event_id`, `file_path`, `file_name`, `size`, `timestamp`	Photos stockées sur le Pi
`room_status`	`status` (libre, occupee, reservee), `last_update`, `reason`	État actuel de la salle
`logs`	`level`, `message`, `source`, `timestamp`	Journal (prévu)
`users`	`username`, `email`, `role`	Comptes (prévu)

Les photos elles-mêmes ne sont pas dans la base : elles sont enregistrées sur la carte SD. La table `images` garde leur chemin, leur taille et leur heure, et elle est reliée à `video_events` par `event_id`. Des index sur `timestamp` accélèrent les recherches par date.

6. API

L'API répond sur `http://172.16.10.239:5000`. Elle renvoie du JSON aux programmes (capteur, caméra, curl) et des pages web aux navigateurs.

Méthode	Adresse	Rôle
GET	`/`	Tableau de bord
GET	`/api/health`	Vérifie que l'API fonctionne
POST	`/api/pir/event`	Enregistre une détection du capteur
GET	`/api/pir/events`	Liste des détections
POST	`/api/video/image`	Reçoit une photo (jpg ou png, 10 Mo maximum)
GET	`/api/images`	Galerie de photos, ou liste JSON
GET	`/api/images/<id>`	Affiche une photo
POST	`/api/video/event`	Enregistre un événement caméra sans photo
GET	`/api/video/events`	Liste des événements caméra
GET	`/api/room/status`	État de la salle

Méthode	Adresse	Rôle
GET	`/api/stats`	Données du tableau de bord

6.1 Format attendu

Le capteur envoie une requête POST en JSON :

```
{"sensor_id": "esp32_pir_01", "duration": 0, "status": "active"}
```

La caméra envoie la photo en `multipart/form-data`, avec les champs `image` (le fichier), `camera_id`, `event_type` et `description`.

6.2 Codes de réponse

Code	Signification
200	Lecture réussie
201	Enregistrement réussi
400	Donnée manquante ou invalide
404	Adresse inconnue
413	Image trop lourde
500	Erreur du serveur ou de la base

7. Raccordement du capteur et de la caméra

Mon serveur devait recevoir les données des deux autres groupes. J'ai fourni à chacun l'adresse de l'API et adapté leur code pour qu'il envoie le bon format.

7.1 Capteur PIR (ESP32)

- Le code ne compilait pas : la variable `serverName` n'existait pas. Je l'ai remplacée par `API_URL`.
- Les champs envoyés (`salle`, `capteur`, `etat`) ne correspondaient pas à l'API. Ils ont été remplacés par `sensor_id`, `duration` et `status`.
- Le capteur envoie `active` à la détection et `inactive` après le délai sans mouvement, avec la durée en secondes.

Résultat dans le moniteur série de l'ESP32 : `Code HTTP : 201` et `event_id: 2`.

7.2 Caméra

- J'ai ajouté au programme de la caméra l'envoi automatique de chaque capture vers `/api/video/image`.
- L'envoi se fait en arrière-plan : la vidéo ne ralentit pas, et la photo reste gardée sur le PC si le serveur ne répond pas.
- La caméra est reliée au PC vidéo par un câble (réseau 192.168.1.0/24), et ce PC est aussi connecté au Wi-Fi `Wifi_Iot215` pour joindre mon serveur.

8. Tests et résultats

Les tests ont été faits avec `curl` depuis un second terminal SSH, puis vérifiés dans la base.

Test	Commande ou action	Résultat
1	`GET /api/health`	`{"status":"running"}` : réussi
2	`POST /api/pir/event`	`event_id: 1`, `status: ok` : réussi
3	`GET /api/pir/events`	Événement listé : réussi
4	`POST /api/video/event`	`event_id: 1`, `status: ok` : réussi
5	`GET /api/video/events`	Événement listé : réussi
6	`GET /api/room/status`	Statut `occupee` après la détection : réussi
7	`SHOW TABLES;` et `SELECT`	6 tables, données présentes : réussi
8	Mouvement devant l'ESP32	`Code HTTP : 201` : réussi
9	Capture de la caméra	Photo visible dans la galerie : réussi
10	Connexion à phpMyAdmin	Tables visibles avec les deux comptes : réussi
11	Sauvegarde `mysqldump`	Fichier `sauvegarde_mp1.sql` créé : réussi

Exemple de test :

```
curl -X POST http://172.16.10.239:5000/api/pir/event \
  -H "Content-Type: application/json" \
  -d '{"sensor_id":"esp32_pir_01","duration":120,"status":"active"}'
```

9. Problèmes rencontrés et solutions

Problème	Cause	Solution
MySQL introuvable	Non fourni pour Raspberry Pi OS	Installation de MariaDB, compatible
`ERROR 1133` à la création de l'utilisateur	Droits donnés à un compte `'<UTILISATEUR_BDD>'@'%'` inexistant	Comptes supprimés puis recréés proprement
`ERROR 1050 Table already exists`	Schéma importé deux fois	Base supprimée et recréée, puis `IF NOT EXISTS` dans le schéma
Environnement `venv` dans `database/`	Créé dans le mauvais dossier	Dossier supprimé
`No module named 'flask'`	API lancée sans activer le `venv`	`source venv/bin/activate`
`can't open file app.py`	Commande lancée depuis le mauvais dossier	`cd ~/mp1-infrastructure/server/api`
`Address already in use`	Une ancienne API tournait encore sur le port 5000	`pkill -f "python3 app.py"`, puis service systemd
`Endpoint not found`	Adresse racine ouverte au lieu d'une route de l'API	Utilisation de `/api/health`
Adresse IP qui change	`dhcpcd.conf` ignoré, NetworkManager utilisé	IP fixe avec `nmcli`
L'API s'arrête avec le terminal	Lancement à la main	Service systemd `mp1-api`
`nginx.service does not exist`	Nginx n'était pas installé	Apache utilisé pour phpMyAdmin

Problème	Cause	Solution
Connexion phpMyAdmin refusée	Mot de passe interne de phpMyAdmin utilisé par erreur	Connexion avec `<UTILISATEUR_BDD>` ou `<ADMIN_BDD>`

10. Accès, sécurité et sauvegarde

10.1 Comptes

Accès	Adresse ou commande	Identifiant	Mot de passe
SSH	`ssh <UTILISATEUR_PI>@172.16.10.239`	`<UTILISATEUR_PI>`	Choisi dans Raspberry Pi Imager
phpMyAdmin (base du projet)	`http://172.16.10.239/phpmyadmin`	`<UTILISATEUR_BDD>`	`<MOT_DE_PASSE_BDD>`
phpMyAdmin (administrateur)	`http://172.16.10.239/phpmyadmin`	`<ADMIN_BDD>`	`<MOT_DE_PASSE_ADMIN>`
MariaDB root	`sudo mysql -u root`	`root`	Par `sudo` uniquement
Tableau de bord et galerie	`http://172.16.10.239:5000/`	Aucun	Aucun

10.2 Sécurité

- Le compte `<UTILISATEUR_BDD>` n'a des droits que sur la base du projet.
- Le mot de passe de la base est dans le fichier `.env`, exclu de GitHub par le `.gitignore`, comme les photos.
- MariaDB n'écoute qu'en local : seule l'API y accède.
- Les fichiers envoyés à l'API sont contrôlés : format jpg ou png, 10 Mo maximum, nom de fichier nettoyé.

10.3 Sauvegarde

```
mysqldump -u <UTILISATEUR_BDD> -p mp1_surveillance > ~/sauvegarde_mp1.sql
```

Le fichier est ensuite copié sur le PC avec le panneau Sftp de MobaXterm. Pour restaurer : `mysql -u <UTILISATEUR_BDD> -p mp1_surveillance < sauvegarde_mp1.sql`.

11. Choix techniques

Choix	Raison
MariaDB au lieu de MySQL	MySQL n'est pas disponible sur le Raspberry Pi ; MariaDB est compatible avec les mêmes commandes.
Apache + phpMyAdmin au lieu de XAMPP	XAMPP n'existe pas pour le processeur ARM du Pi ; ce sont les mêmes logiciels installés séparément.
Pas de Nginx	Apache occupe déjà le port 80 pour phpMyAdmin. Un second serveur web aurait été en conflit. L'API reste accessible sur le port 5000, et Apache pourrait servir de reverse proxy avec `mod_proxy` si besoin.
Flask	Framework Python léger, adapté au Raspberry Pi et simple à faire évoluer.

Choix	Raison
Photos sur disque, chemin en base	Une base garde de petites données rapides à chercher ; les fichiers images restent sur la carte SD.
Service systemd	Démarrage automatique et redémarrage en cas d'arrêt, sans terminal ouvert.

12. Bilan

12.1 Ce qui fonctionne

- Serveur Raspberry Pi avec adresse fixe et API qui démarre seule.
- Base MariaDB gérée avec phpMyAdmin.
- Réception des détections du capteur et des photos de la caméra.
- Tableau de bord graphique et galerie de photos avec date et heure.
- Sauvegarde de la base.

12.2 Améliorations possibles

- Activer le pare-feu UFW en n'ouvrant que les ports 22, 80 et 5000 au réseau 172.16.10.0/24.
- Ajouter une authentification sur l'API et le tableau de bord (table `users` déjà prévue).
- Automatiser la sauvegarde chaque jour avec `cron`.
- Supprimer automatiquement les photos les plus anciennes pour ne pas remplir la carte SD.

12.3 Ce que j'ai appris

J'ai appris à installer et administrer un serveur Linux à distance, à configurer une adresse IP fixe, à créer une base de données et une API web, et à rendre un service automatique avec systemd. J'ai aussi appris à diagnostiquer des erreurs à partir de leurs messages, et à faire communiquer des appareils différents grâce à un format de données commun.